

■ Striker Performance Analysis Using Python

```
# Load Packages
```

Code Cell 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import f_oneway
from scipy.stats import shapiro
from scipy.stats import levene
from scipy.stats import pearsonr
import statsmodels.api as sm
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

At this stage, essential Python libraries are imported — including tools for data handling, visualization, and analysis. These imports form the foundation for the rest of the project.

```
# Load Dataset
```

Code Cell 3

```
data = pd.read_excel('Strikers_performance.xlsx')
data.head()
```

Here, the dataset is loaded into a pandas DataFrame from an Excel file that contains the performance metrics of football strikers.

```
# Data Cleaning
```

```
## Missing values
```

Code Cell 6

```
data.isnull().sum()
```

This step focuses on cleaning the dataset by checking and handling missing values using imputation.

Code Cell 7

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='median')
data[['Movement off the Ball',
      'Big Game Performance',
      'Penalty Success Rate']] = imputer.fit_transform(data[['Movement off the Ball',
                                                              'Big Game Performance',
                                                              'Penalty Success Rate']])

data.isnull().sum()
```

At this stage, essential Python libraries are imported — including tools for data handling, visualization, and analysis. These imports form the foundation for the rest of the project.

```
## Data Types
```

Code Cell 9

```
data.dtypes
```

The data types of columns are reviewed and adjusted to ensure proper analysis and modeling later.

Code Cell 10

```
variables = ['Goals Scored', 'Assists',  
            'Shots on Target',  
            'Movement off the Ball',  
            'Hold-up Play',  
            'Aerial Duels Won',  
            'Defensive Contribution',  
            'Big Game Performance',  
            'Impact on Team Performance',  
            'Off-field Conduct']
```

```
for var in variables:  
    data[var] = data[var].astype('int')
```

```
data.dtypes
```

The data types of columns are reviewed and adjusted to ensure proper analysis and modeling later.

Code Cell 11

```
data.head()
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

```
# Exploratory Data Analysis
```

```
#### Perform descriptive analysis
```

Code Cell 14

```
round(data.describe(), 2)
```

Descriptive statistics are generated to provide an overview of the numerical features in the dataset.

```
#### Perform percentage analysis
```

Code Cell 16

```
freq_Footedness = data['Footedness'].value_counts()  
perc_Footedness = freq_Footedness/len(data['Footedness'])*100  
perc_Footedness
```

Frequency and percentage analysis help in understanding categorical distributions such as player footedness.

Code Cell 17

```
plt.figure(figsize=(12, 6))  
perc_Footedness.plot(kind='pie', autopct='%1.2f%%')  
plt.title('Percentage of strikers by their footedness')  
plt.ylabel('')  
plt.show()
```

This section creates visualizations using Matplotlib or Seaborn to highlight patterns in striker performance.

Which nationality strikers have the highest average number of goals scored?

Code Cell 19

```
goals_by_nationality = data.groupby('Nationality')['Goals Scored'].mean().sort_values(ascending=False)
round(goals_by_nationality)
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

What is the average conversion rate for players based on their footedness?

Code Cell 21

```
conversion_rate_by_footedness = data.groupby('Footedness')['Conversion Rate'].mean()
conversion_rate_by_footedness
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

What is the distribution of players' footedness across different nationalities?

Code Cell 23

```
footedness_by_nationality = pd.crosstab(data['Nationality'], data['Footedness'])
footedness_by_nationality
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Code Cell 24

```
plt.figure(figsize=(12, 6))
sns.countplot(x='Nationality', hue='Footedness', data=data)
plt.title('Distribution of players footedness across different nationalities')
plt.xlabel('Nationality')
plt.ylabel('Count')
plt.show()
```

This section creates visualizations using Matplotlib or Seaborn to highlight patterns in striker performance.

Create a correlation matrix with a heatmap

Code Cell 26

```
num_variables = data.select_dtypes(include = ['number']).columns

correl_matrix = round(data[num_variables].corr(), 3)
correl_matrix
```

The data types of columns are reviewed and adjusted to ensure proper analysis and modeling later.

Code Cell 27

```
plt.figure(figsize=(18, 10))
sns.heatmap(correl_matrix, annot=True)
plt.title('Heatmap of Correlation Matrix')
plt.show()
```

This section creates visualizations using Matplotlib or Seaborn to highlight patterns in striker performance.

Statistical Test

Find whether there is any significant difference in consistency rates among strikers from various nationality

Code Cell 30

```
# Normality test
stat, p_value = shapiro(data['Consistency'])
print('P value: ', round(p_value, 3))
```

Statistical tests like ANOVA and Levene's test are performed to validate assumptions about data variance and normality.

Code Cell 31

```
# Filtering data
Spain = data.query('Nationality == "Spain"')['Consistency']
France = data.query('Nationality == "France"')['Consistency']
Germany = data.query('Nationality == "Germany"')['Consistency']
Brazil = data.query('Nationality == "Brazil"')['Consistency']
England = data.query('Nationality == "England"')['Consistency']
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Code Cell 32

```
# Levene test for statistics
stats, p_value = levene(Spain, France, Germany, Brazil, England)
print("P value: ", round(p_value, 3))
```

Statistical tests like ANOVA and Levene's test are performed to validate assumptions about data variance and normality.

Code Cell 33

```
# One way ANOVA
Test_stat, p_value = f_oneway(Spain, France, Germany, Brazil, England)
print("P value: ", round(p_value, 2))
```

Statistical tests like ANOVA and Levene's test are performed to validate assumptions about data variance and normality.

Check if there is any significant correlation between strikers' Hold-up play and consistency rate

Code Cell 35

```
# Normality test
stat, p_value = shapiro(data['Hold-up Play'])
print('P value: ', round(p_value, 3))
```

Statistical tests like ANOVA and Levene's test are performed to validate assumptions about data variance and normality.

Code Cell 36

```
# Linearity test
plt.figure(figsize = (10, 6))
sns.regplot(x = 'Hold-up Play', y = 'Consistency', data = data)
plt.title('Linearity between Hold-up Play and Consistency')
plt.xlabel('Hold-up Play')
plt.ylabel('Consistency')
plt.show()
```

This section creates visualizations using Matplotlib or Seaborn to highlight patterns in striker performance.

Code Cell 37

```
# Pearson correlation
HU_play = data['Hold-up Play']
Consistency = data['Consistency']

corr, p_value = pearsonr(HU_play, Consistency)
print("Correlation coefficient: ", round(corr, 3))
print("P value: ", round(p_value, 3))
```

Correlation analysis is conducted to measure relationships between numerical variables such as goals and assists.

Check if strikers' hold-up play significantly influences their consistency rate

Code Cell 39

```
x = data['Hold-up Play']
y = data['Consistency']

x_constant = sm.add_constant(x)
model = sm.OLS(y, x_constant).fit()

print(model.summary())
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Feature Engineering

Create a new feature - Total contribution score

Code Cell 42

```
data['Total contribution score'] = (data['Goals Scored'] + data['Assists'] + data['Shots on Target'] + 
data.head()
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Encode the Footedness and marital status by LabelEncoder

Code Cell 44

```
encoder = LabelEncoder()
data['Footedness'] = encoder.fit_transform(data['Footedness'])
data['Marital Status'] = encoder.fit_transform(data['Marital Status'])
```

```
data.head()
```

Data preprocessing is done here — categorical values are encoded and numerical features standardized for fair model training.

```
#### Create the dummies for Nationality and add with the data
```

Code Cell 46

```
dummies = pd.get_dummies(data['Nationality'])
processed_df = pd.concat([data, dummies], axis = 1)
processed_df = processed_df.drop('Nationality', axis = 1)
processed_df.head()
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

```
# Cluster Analysis
```

```
#### Perform KMeans clustering
```

Code Cell 49

```
# Selecting features
x = processed_df.drop('Striker_ID', axis = 1)

# Calculating WCSS score
wcss = []

for i in range(1, 15):
    kmeans = KMeans(n_clusters = i, init = 'k-means++')
    kmeans.fit(x)
    wcss_score = kmeans.inertia_
    wcss.append(wcss_score)
```

This part applies the K-Means clustering algorithm to group strikers into categories based on performance.

Code Cell 50

```
# Plotting elbow chart
plt.figure(figsize = (12, 6))
plt.plot(range(1, 15), wcss, marker = 'o')
plt.title('Elbow methods')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS')
plt.show()
```

This section creates visualizations using Matplotlib or Seaborn to highlight patterns in striker performance.

Code Cell 51

```
# Building KMeans with k = 2
final_km = KMeans(n_clusters = 2)
final_km.fit(x)

# Generating labels
labels = final_km.labels_
labels
```

This part applies the K-Means clustering algorithm to group strikers into categories based on performance.

Code Cell 52

```
# Adding labels
processed_df['Clusters'] = labels
processed_df.head()
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Code Cell 53

```
# Checking clusters
round(processed_df.groupby('Clusters')['Total contribution score'].mean(), 2)
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Code Cell 54

```
# Assigning meaningful names
mapping = {0:'Best strikers', 1:'Regular strikers'}
processed_df['Strikers types'] = processed_df['Clusters'].map(mapping)
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Code Cell 55

```
# Deleting the Clusters variable
processed_df = processed_df.drop('Clusters', axis = 1)
processed_df.head()
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Data Preprocessing for ML

New feature mapping

Code Cell 58

```
mapping = {'Best strikers':1, 'Regular strikers': 0}
processed_df['Strikers types'] = processed_df['Strikers types'].map(mapping)
processed_df.head()
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

Selecting features

Code Cell 60

```
x = processed_df.drop(['Striker_ID', 'Strikers types'], axis = 1)
y = processed_df['Strikers types']
```

This step performs data manipulation or analysis contributing to the overall understanding of striker performance.

```
### Scaling features
```

Code Cell 62

```
scaler = StandardScaler()
scaled_x = scaler.fit_transform(x)
scaled_x
```

Data preprocessing is done here — categorical values are encoded and numerical features standardized for fair model training.

```
### Train test split
```

Code Cell 64

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(scaled_x, y, test_size = 0.2, random_state = 42)
```

At this stage, essential Python libraries are imported — including tools for data handling, visualization, and analysis. These imports form the foundation for the rest of the project.

```
# Predictive Classification Analytics
```

```
### Build a logistic regression machine learning model to predict strikers type
```

Code Cell 67

```
# Model training
lgr_model = LogisticRegression()
lgr_model.fit(x_train, y_train)

#Prediction
y_lgr_pred = lgr_model.predict(x_test)

# Evaluation
accuracy_lgr = accuracy_score(y_test, y_lgr_pred)
print(accuracy_lgr*100, '%')
```

A Logistic Regression model is trained and evaluated to predict striker performance categories.

Code Cell 68

```
# Creating confusion matrix
conf_matrix_lgr = confusion_matrix(y_test, y_lgr_pred)

# Plotting confusion matrix
plt.figure(figsize = (12, 6))
sns.heatmap(conf_matrix_lgr, annot = True, fmt = "d", cmap = "Blues")
plt.title('Confusion Matrix for LGR model')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

This section creates visualizations using Matplotlib or Seaborn to highlight patterns in striker performance.

```
## Thank you!
```

